

Mundo Escolar — Documentación técnica

Plataforma educativa de escritorio para estudiantes de **1° a 6° grado de primaria**. Reúne cinco materias en una sola aplicación: Lectura y Escritura, Ajedrez, Computación, Robótica y Dibujo. Está desarrollada en **Python** con interfaz **CustomTkinter**, y algunos minijuegos usan **Pygame** para actividades más intensivas (ajedrez, mecanografía, precisión del mouse).

Tabla de contenidos

1. ¿Qué hace el software?
2. Estructura del proyecto
3. Arquitectura general
4. Patrones de diseño y principios
5. El núcleo (`core/`)
6. Punto de entrada: `main.py`
7. Flujo de datos: vista ↔ lógica ↔ persistencia
8. Los cinco módulos educativos
9. Sistema de logros y progreso
10. Cómo añadir un módulo nuevo
11. Ejecución y dependencias

¿Qué hace el software?

Mundo Escolar funciona como un **hub central** (menú principal) desde el que el niño:

1. **Elige su perfil** (nombre de usuario con persistencia en JSON).
2. **Navega** por cinco áreas temáticas mediante tarjetas visuales.
3. **Juega y aprende** en minijuegos adaptados a primaria.
4. **Acumula progreso** por materia (barra de 0 % a 100 %).
5. **Desbloquea logros** por acciones concretas o por porcentaje de avance.
6. **Consulta** su progreso global y su colección de logros desde el menú lateral.

La aplicación arranca en **pantalla completa**, con barra lateral colapsable, temas visuales distintos por módulo y diseño **responsive** (fuentes y tamaños se adaptan al ancho de pantalla).

Estructura del proyecto

```
Mundo_Escolar/
├── main.py                # Shell de la aplicación: UI global, navegación, usuarios
├── requirements.txt       # Dependencias Python
├── mundo-escolar.sh      # Script de arranque (Linux / ChromeOS)
├── mundo-escolar.desktop # Entrada de escritorio
├──
├── core/                 # Servicios transversales compartidos por todos los módulos
│   ├── gestor_estado.py  # Persistencia del progreso (Singleton)
│   ├── module_factory.py # Registro y carga dinámica de módulos (Factory)
│   ├── module_manager.py # Ventana/módulo activo y cierre con Escape (Singleton)
│   ├── theme.py          # Colores, fuentes y tamaños responsivos
│   ├── logros_config.py  # Catálogo declarativo de logros
│   ├── util_logros.py    # Modal de celebración al desbloquear logros
│   ├── admin_tools.py    # Panel docente (Ctrl+Shift+A)
│   └── data/usuarios/    # JSON por estudiante (no versionado en git)
├──
├── modulos/              # Cinco módulos educativos independientes
│   ├── Lenguaje/         # Lectura y Escritura
│   ├── Ajedrez/
│   ├── Computacion/     # Incluye mecanografía (typing) y mouse
│   ├── Robotica/
│   └── Dibujo/
├──
├── assets/               # Recursos globales (logo, tarjetas del menú, fuente)
│   ├── imagenes/
│   └── Super Meatball.ttf
├──
├── ui/
│   └── helpers.py        # Utilidades UI compartidas (navbar legacy)
├──
└── lib/                  # Librerías locales opcionales (Linux/ChromeOS)
```

Los cinco módulos registrados

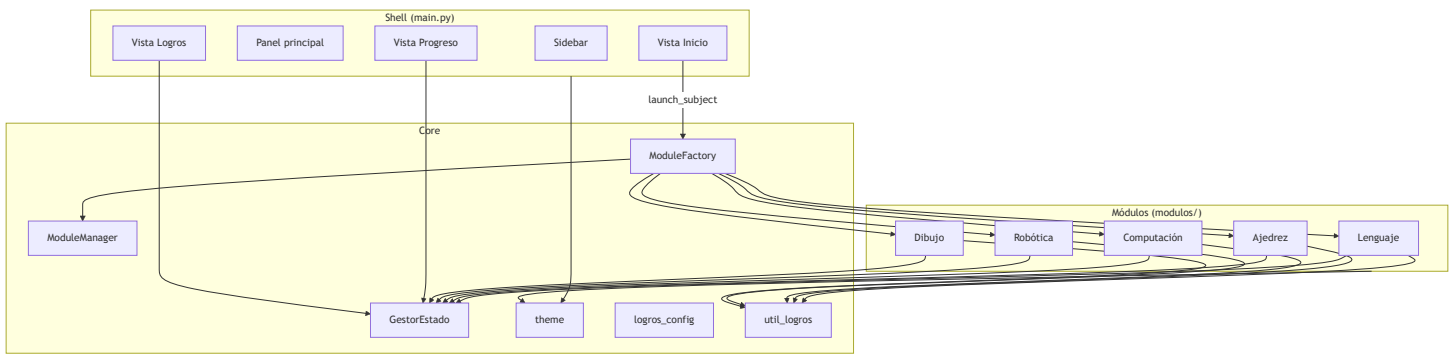
ID en factory	Carpeta	Nombre en pantalla	Tecnología principal
lectura_escritura	Lenguaje/	Lectura y Escritura	CustomTkinter
ajedrez	Ajedrez/	Ajedrez	CTk + Pygame + Stockfish
computacion	Computacion/	Computación	CTk + Pygame
robotica	Robotica/	Robotica	CustomTkinter embebido
dibujo	Dibujo/	Dibujo	CTk + Canvas Tkinter

Nota: No existe un módulo independiente llamado "Typing" ni "Lectura". La mecanografía vive dentro de **Computación**; la lectura/escritura vive en el paquete **Lenguaje**.

Arquitectura general

El diseño sigue un modelo **hub-and-spoke** (centro y radios):

- **main.py** es el **hub**: navegación, usuario, progreso global, logros y lanzamiento de módulos.
- **core/** provee **servicios transversales**: estado, fábrica de módulos, tema, logros.
- Cada carpeta en **modulos/** es un **spoke** independiente que expone un único contrato de entrada.



Arquitectura por capas (modelo conceptual)

Aunque no hay un framework MVC formal, cada módulo sigue una **separación por capas** reconocible:

Capa	Responsabilidad	Ubicación típica
Presentación / Vista	Widgets, eventos, animaciones, lo que ve el usuario	vistas/ , views/ , *_menu.py , main.py
Dominio / Lógica	Reglas de juego sin dependencias gráficas	dominio/ , domain/ , robotica_dominio.py
Servicios	Audio, IA, datos externos, utilidades	servicios/ , services/
Datos	Preguntas, textos, assets, configuración	servicios/preguntas.py , domain/texts.py , config/ , assets/
Infraestructura	Persistencia, logros, tema, carga de módulos	core/

Dos niveles de ventana

Nivel	Tecnología	Ejemplos
Shell global	CustomTkinter embebido en <code>main.principal</code>	Menú principal, menús de módulo, robótica
Juegos individuales	CTkToplevel fullscreen o Pygame	JuegoBase (Lenguaje), VentanaJuego (Ajedrez), TypingModule

El shell **no conoce** la lógica interna de los juegos; solo el contrato `crear_frame(parent, on_back)` y las llamadas al `core` .

Patrones de diseño y principios

Patrones identificados

Patrón	Archivo(s)	Descripción
Singleton	GestorEstado , ModuleManager	Una única instancia global vía <code>__new__</code> y <code>_instancia</code>

Patrón	Archivo(s)	Descripción
Factory Method	ModuleFactory	<code>get_module_frame()</code> crea el módulo según <code>subject_id</code>
Registry / Configuration	SUBJECTS , LOGROS , MODULO_THEMES	Datos declarativos desacoplados del código
Facade	<code>main.py</code> , <code>*_menu.py</code>	Interfaz simple (<code>crear_frame</code>) sobre subsistemas complejos
Template Method	JuegoBase (Lenguaje), BaseModule (Computación)	Estructura común; subclasses implementan el juego
Strategy (ligero)	<code>aplicar_tema_interfaz()</code>	Intercambia paleta visual según el módulo activo

Principios SOLID (aplicación práctica)

Principio	Cómo se refleja en el proyecto
S — Responsabilidad única	<code>gestor_estado</code> solo persiste; <code>module_factory</code> solo carga módulos; <code>theme</code> solo define apariencia
O — Abierto/cerrado	Nuevos módulos se añaden registrándolos en <code>SUBJECTS</code> sin modificar la lógica de carga
L — Sustitución de Liskov	Todos los módulos cumplen <code>crear_frame(parent, on_back) -> CTKFrame</code>
I — Segregación de interfaces	Contrato mínimo: un solo callable por módulo; no se fuerza herencia común
D — Inversión de dependencias	Los módulos dependen de abstracciones del <code>core</code> (<code>GestorEstado</code> , <code>theme</code>), no de <code>main.py</code> directamente

Convenciones de nombres

- **Lenguaje y Robótica** usan carpetas en español: `vistas/` , `dominio/` , `servicios/` .
- **Ajedrez y Computación** usan inglés: `views/` , `domain/` , `services/` .
- Ambas convenciones coexisten; la integración es idéntica vía `ModuleFactory` .

El núcleo (core/)

gestor_estado.py — Singleton de persistencia

Responsabilidad: Guardar y cargar el progreso de cada estudiante en `core/data/usuarios/{nombre}.json` .

Estado por defecto de cada usuario:

```
{
  "materias": {
    "Lectura y Escritura": 0.0,
    "Ajedrez": 0.0,
    "Computación": 0.0,
    "Robotica": 0.0,
    "Dibujo": 0.0,
  },
  "logros_obtenidos": [],
  "robotica": {
    "misiones_completadas": 0,
    "monedas": 0,
    "niveles_completados": [],
    "dificultad": "basico",
  },
}
```

API principal:

Método	Uso
<code>inicializar_usuario(nombre)</code>	Carga o crea el JSON del usuario
<code>guardar()</code>	Persiste datos y logros
<code>reset()</code>	Vuelve al estado por defecto
<code>actualizar_progreso(materia, incremento)</code>	Suma avance (máx. 1.0); sincroniza logros de porcentaje
<code>desbloquear_logro(id)</code> / <code>tiene_logro(id)</code>	Gestión de logros
<code>get_materias()</code>	Dict de progreso por materia

Método	Uso
<code>limpiar_si_es_necesario()</code>	Borra todos los JSON cada 7 días

Los nombres de materia deben coincidir **exactamente** con los definidos aquí y en `logros_config.py` .

module_factory.py — Factory de módulos

Responsabilidad: Catálogo declarativo (`SUBJECTS`) e importación dinámica de cada módulo.

Contrato obligatorio de cada módulo:

```
def crear_frame(parent, on_back_callback) -> CTkFrame:
    ...
```

Flujo de `get_module_frame(subject_id, parent, on_back_callback)` :

1. Busca la configuración en `SUBJECTS` .
2. Importa el módulo con `__import__(import_path)` .
3. Obtiene y ejecuta `crear_frame(parent, on_back_callback)` .
4. Asigna `_volver_callback` y enlaza `<Escape>` al callback de vuelta.
5. Registra el frame en `ModuleManager().open_module(frame)` .

Helpers globales:

- `set_main_root(root)` — llamado desde `main` al crear la ventana.
- `get_main_root()` — usado por módulos y `util_logros` para diálogos modales.

module_manager.py — Singleton de módulo activo

Responsabilidad: Registrar la ventana o frame del módulo activo y unificar el cierre con `Escape` o destrucción del widget.

- `open_module(window)` — cierra el anterior, registra el nuevo, configura atajos.
- `close_module()` — invoca `WM_DELETE_WINDOW` si existe (importante para liberar hilos de Pygame en Ajedrez).

theme.py — Tema visual compartido

Responsabilidad: Paleta `COLORS` , fuente `FONT_NAME` ("Super Meatball" o fallback), diccionario `FONTS` responsivo y función `tamano_tarjeta()` .

- `fuentes_responsivas(ancho)` escala entre 1200 px y 2560 px.
- Usado por `main.py` y todos los módulos para coherencia visual.

logros_config.py — Catálogo de logros

Lista declarativa `LOGROS` (~40 entradas). Cada logro tiene: `id` , `nombre` , `desc` , `tipo` , `link` , `req` , `color` .

Tipo	Criterio de desbloqueo
porcentaje	<code>materias[link] >= req</code> (ej. 25 %, 60 %, 100 %)
accion	Solo vía <code>notificar_logro(id)</code> desde un juego
general	3 materias al 100 % o todas las materias

util_logros.py — Notificación visual de logros

`notificar_logro(logro_id, parent=None) -> bool`

1. Llama a `GestorEstado().desbloquear_logro(logro_id)` .
2. Si es nuevo, muestra `VentanaLogroCustom` (modal de celebración).
3. Si `parent` es `None` , usa `get_main_root()` .

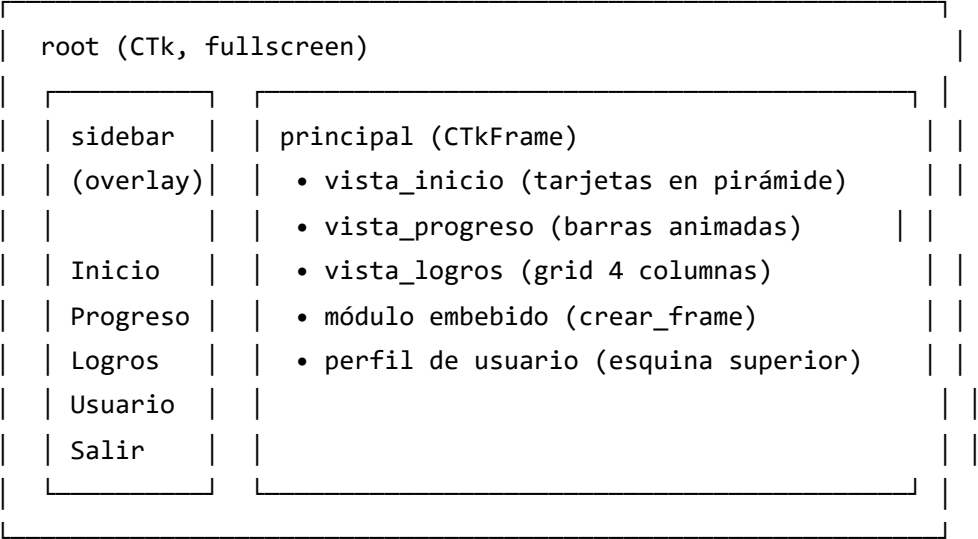
admin_tools.py — Panel docente

Clase `AdminTools(ctk.CTkToplevel)` en pantalla completa. Permite listar alumnos, eliminar perfiles y borrar todos los progresos. Atajo: **Ctrl+Shift+A** .

Punto de entrada: main.py

main.py es el **shell de la aplicación**: ventana principal, barra lateral, panel central embebido, gestión de usuario, progreso, logros y lanzamiento de módulos.

Layout de la interfaz



- **Sidebar:** place(x=0, y=0, relheight=1) como overlay; no desplaza el contenido al colapsar.
- **Principal:** pack(expand=True, fill="both", padx=(ancho_sidebar, 0)) con margen fijo.
- **Fullscreen** por defecto; Escape sale de fullscreen (no cierra la app).
- **Responsive:** _on_resize → actualizar_fuentes_responsivas() + ajustar_sidebar() .

Variables de estado global

Variable	Rol
vista_actual	Referencia al CtkFrame del módulo o vista activa
_loading_module	Evita doble clic al cargar módulos
_tarjetas_deshabilitadas	Bloquea tarjetas del inicio durante la carga
perfil_frame_ref	Widget del indicador de usuario
sidebar_expandido	Estado colapsado/expandido del sidebar

Variable	Rol
botones_menu	Dict {nombre: CtkButton} para resaltar vista activa

Flujo de arranque

- 1. Crear root Ctk fullscreen
- 2. set_main_root(root)
- 3. GestorEstado().limpiar_si_es_necesario()
- 4. solicitar_usuario() → diálogo modal obligatorio
- 5. Construir sidebar + botones de menú
- 6. vista_inicio() → pantalla de tarjetas
- 7. root.mainloop()

Vistas del panel central

Función	Qué muestra el usuario
vista_inicio()	Logo + 5 tarjetas en pirámide invertida (3 arriba, 2 abajo). Cada tarjeta abre un módulo.
vista_progreso()	Barras animadas por materia + promedio global del estudiante activo.
vista_logros()	Grid de 4 columnas con tarjetas de logros (bloqueados/desbloqueados).
launch_subject(id)	Carga el módulo embebido y aplica su tema visual.

Temas inmersivos por módulo

MODULO_THEMES define paletas (bg , sidebar , border , text , hover) por subject_id . La función aplicar_tema_interfaz(tema_id) actualiza colores del sidebar, botones y fondo. En **Dibujo** también cambia el cursor a lápiz. Al volver a inicio se restaura "default" .

Funciones clave de navegación

Función	Comportamiento
limpiar_panel()	Destruye vista_actual y todos los hijos de principal
launch_subject(subject_id)	Limpia → aplica tema → ModuleFactory.get_module_frame() → +5 % progreso

Función	Comportamiento
<code>vista_inicio()</code>	Limpia → tema default → perfil usuario → tarjetas → muestra botón Salir

Diálogos modales

- `VentanaUsuarioCustom` — registro/selección de nombre al iniciar o crear perfil nuevo.
- `VentanaSalirCustom` — confirmación de salida de la aplicación.

Flujo de datos: vista ↔ lógica ↔ persistencia

Secuencia al abrir un módulo

```

Usuario hace clic en tarjeta
→ launch_subject("ajedrez")
→ limpiar_panel()
→ aplicar_tema_interfaz("ajedrez")
→ ModuleFactory.get_module_frame("ajedrez", principal, vista_inicio)
→ import modulos.Ajedrez.main_ajedrez
→ crear_frame(principal, vista_inicio)
→ ModuleManager.open_module(frame)
→ GestorEstado().actualizar_progreso("Ajedrez", 0.05)
→ frame.pack(fill="both", expand=True)

```

Secuencia al completar una actividad (ejemplo: Lenguaje)

```

VistaDetectiveSonidos (vista)
→ DetectiveSonidos.verificar() (dominio)
→ gestor.actualizar_progreso("Lectura y Escritura", incremento)
→ notificar_logro("lenguaje_nivel_1", parent=self)
→ GestorEstado.desbloquear_logro()
→ VentanaLogroCustom (modal)

```

Puentes comunes entre capas

Puente	Uso
GestorEstado()	Lectura/escritura de progreso y logros
notificar_logro(id, parent)	Logros por acción con modal de celebración
from core.theme import COLORS, FONTS	Consistencia visual
on_back_callback	Navegación de vuelta al menú sin imports circulares
get_main_root()	Ventana padre para diálogos desde hilos o lógica sin widget directo

Los cinco módulos educativos

1. Lenguaje — Lectura y Escritura

Entrada: `modulos/Lenguaje/lenguaje_menu.py` → `crear_frame`

ID factory: `lectura_escritura`

Estructura de archivos

```
modulos/Lenguaje/
├─ lenguaje_menu.py      ← Menú mapa de niveles (punto de entrada)
├─ dominio/
│   └─ juegos.py        ← Lógica pura (sin tkinter)
├─ servicios/
│   ├── preguntas.py    ← Banco de preguntas y contenido educativo
│   ├── escribe_palabra.py
│   └─ audio.py         ← Reproducción de sonidos MP3
├─ vistas/
│   ├── base.py         ← JuegoBase (CTkToplevel fullscreen)
│   ├── detective_sonidos.py
│   ├── constructor_palabras.py
│   ├── carrera_letras.py
│   ├── escribe_palabra.py
│   ├── rimas.py
│   ├── ordena_letras.py
│   ├── lee_encuentra.py
│   └─ sopa_letras.py
└─ assets/
    ├── *.png           ← Iconos del menú
    └─ Sonidos/         ← MP3 por emoji/letra
```

Capas

Capa	Archivos	Rol
Vista	lenguaje_menu.py , vistas/*.py	Menú mapa + 8 minijuegos fullscreen
Dominio	dominio/juegos.py	DetectiveSonidos , ConstructorPalabras , SopaLetras , etc.
Servicios	servicios/preguntas.py , audio.py	Datos y reproducción de audio
Core	GestorEstado , util_logros	Progreso y logros por nivel

Qué ve el usuario

- 1. **Menú mapa** estilo pergamino/madera con 8 botones en un camino curvo.
- 2. Al pulsar un nivel → **ventana fullscreen** del minijuego:

#	Minijuego	Actividad
1	Detective de Sonidos	Escuchar letra → elegir emoji correcto
2	Constructor	Unir sílabas para formar palabras
3	Carrera de Letras	Diferenciar letras parecidas (b/d, p/q)
4	Escribe Palabras	Ver imagen → escribir el nombre
5	Rimas Divertidas	Encontrar palabras que riman
6	Ordena Letras	Ordenar letras desordenadas
7	Lee y Encuentra	Leer palabra → elegir dibujo
8	Sopa de Letras	Buscar palabras en cuadrícula

Integración

- JuegoBase (Template Method) provee cabecera, contenedor y botón volver.
- Cada vista instancia su clase de dominio y delega la verificación.
- Logros: `lenguaje_nivel_1 ... lenguaje_nivel_8`.

2. Ajedrez

Entrada: `modulos/Ajedrez/main_ajedrez.py` → `crear_frame`

ID factory: `ajedrez`

Estructura de archivos

```
modulos/Ajedrez/  
├─ main_ajedrez.py          ← Menú de configuración (punto de entrada)  
├─ tutorial_ajedrez.py      ← VentanaTutorial (Pygame)  
├─ config/  
│   └─ tutorial_config.py   ← LESSONS: lecciones del tutorial  
├─ domain/  
│   ├── pieza.py  
│   ├── tablero_logico.py   ← Reglas puras (movimientos, jaque, enroque)  
│   ├── partida.py          ← Estado de partida  
│   └─ partida_tutorial.py  
├─ services/  
│   ├── motor_ia.py         ← Wrapper Stockfish (python-chess)  
│   └─ audio.py  
├─ views/  
│   ├── ventana_juego.py    ← Bucle Pygame de partida  
│   ├── tablero_render.py   ← Render del tablero  
│   ├── pieza_animada.py  
│   ├── tutorial_menu.py     ← Academia de Cadetes (CTkToplevel)  
│   ├── tutorial_panel_render.py  
│   ├── tutorial_view_render.py  
│   └─ Flecha.py           ← Flechas tácticas en tablero  
└─ assets/  
    ├── piezas_animadas/    ← PNG por pieza y color  
    ├── imagenes/           ← victoria, derrota, trofeo, etc.  
    ├── stockfish/  
    └─ stockfish linux/     ← Motor Linux/ChromeOS
```

Capas

Capa	Archivos	Rol
Vista menú	main_ajedrez.py , tutorial_menu.py	Configuración de partida, logros
Vista juego	ventana_juego.py , renders	Pygame fullscreen
Dominio	tablero_logico.py , partida.py	Reglas sin dependencias gráficas
Servicios	motor_ia.py , audio.py	Stockfish + sonidos

Qué ve el usuario

1. **Menú espacial** (azul medianoche + cian): elegir bando, dificultad (12+ rangos), tiempo, tema, modo IA/amigo.
2. **Partida Pygame fullscreen**: tablero animado, relojes, capturas, IA con nicknames por nivel.
3. **Tutorial "Academia de Cadetes"**: lecciones guiadas (movimientos, jaque, clavada, etc.).
4. **Panel de logros "Salón de la Estrategia"**: rangos de IA conquistados.

Integración

- Pygame corre en **hilo daemon** con `threading.Event` para cierre seguro.
- `ModuleManager` respeta `WM_DELETE_WINDOW` al cerrar.
- Logros por rango: `ajedrez_hierro ... ajedrez_místico`.

3. Computación (mecanografía + mouse)

Entrada: `modulos/Computacion/computacion_menu.py` → `crear_frame`

ID factory: `computacion`

La mecanografía (`TypingModule`) no es un módulo aparte; vive en `services/game_utils.py`.

Estructura de archivos

```
modulos/Computacion/
├─ computacion_menu.py      ← PanelComputacion (punto de entrada)
├─ config/
│   └─ rutas.py             ← Ruta fuente Super Meatball
├─ domain/
│   ├─ base_module.py       ← ABC BaseModule.run()
│   └─ texts.py             ← LIBROS_MECANOGRAFIA (cuentos por capítulos)
├─ services/
│   ├─ game_utils.py        ← TypingModule (Pygame)
│   ├─ logic_utils.py       ← ComboManager, VisualFeedbackManager
│   └─ tmp_typing_test.py
└─ views/
    ├─ mouse_module.py      ← MouseModule (Pygame, 20 niveles)
    └─ ui_components.py     ← Botón reutilizable Pygame
```

Capas

Capa	Archivos	Rol
Vista menú	computacion_menu.py	Terminal cyber con tarjetas de actividades
Vista juegos	game_utils.py , mouse_module.py	Pygame fullscreen
Dominio	texts.py , logic_utils.py	Textos educativos, combos, feedback
Servicios	Hilos daemon	Ejecución de juegos sin bloquear CTK

Qué ve el usuario

1. **Terminal de Computación** estilo cyber/neón.
2. **Mecanografía**: elegir libro (El Principito, Don Quijote, Cerditos, Pinocho) → escribir fragmentos con retroalimentación de color.
3. **Precisión Mouse**: 20 niveles de clic rápido (izquierdo/derecho) con dificultad escalada.
4. Banner "¿Sabías qué?" con datos curiosos.
5. Panel de logros por biblioteca completada.

Integración

- BaseModule define el contrato run() para juegos Pygame.
- Al cerrar Pygame: actualizar_progreso("Computación", 0.05) .
- Logros: computacion_principito , computacion_don_quijote , etc.

4. Robótica

Entrada: modulos/Robotica/robotica_menu.py → crear_frame

ID factory: robotica

Estructura de archivos

```
modulos/Robotica/  
├─ robotica_menu.py          ← PanelRobotica (punto de entrada)  
├─ robotica_dominio.py      ← Lógica de las 3 misiones  
├─ servicios/  
│   ├─ preguntas_robotica.py ← Preguntas, tarjetas, datos curiosos  
│   └─ audio.py  
└─ vistas/  
    ├─ conoce_robot.py      ← Misión 1: partes del robot  
    ├─ sensores_actuadores.py ← Misión 2: emparejar  
    └─ programacion.py      ← Misión 3: rutas con flechas
```

Capas

Capa	Archivos	Rol
Vista menú	robotica_menu.py	Centro de Mando Robótico
Vista misiones	vistas/*.py	CTk embebido (sin Pygame)
Dominio	robotica_dominio.py	JuegoPartesRobot , JuegoEmparejar , RobotProgramable
Datos	preguntas_robotica.py	Preguntas y obstáculos por dificultad

Qué ve el usuario

1. **Centro de Mando** con perfil de piloto y 3 tarjetas de misión.
2. **Misión 1** — Partes del Robot: imagen + opciones múltiples con timer.
3. **Misión 2** — Empareja y Aprende: componente ↔ función.
4. **Misión 3** — Sigue el Camino: programar ruta con flechas en laberinto.
5. Banner de dato curioso y panel "Registro de Misiones".

Integración

- Misiones se abren **dentro del mismo frame** (_abrir_mision destruye hijos y embebe la vista).
- Sub-estado en JSON: robotica.misiones_completadas , monedas, niveles.
- Logros: robotica_partes , robotica_emparejar , robotica_programacion .

5. Dibujo

Entrada: `modulos/Dibujo/vistas/app/paint_app.py` → `crear_frame`

ID factory: `dibujo`

`dibujo_menu.py` existe como menú legacy en `Toplevel` independiente, pero **no** es el punto de entrada del factory.

Estructura de archivos

```
modulos/Dibujo/
├─ dibujo_menu.py      ← Legacy (no usado por ModuleFactory)
├─ vistas/
│   └─ app/
│       └─ paint_app.py  ← ENTRADA + PaintApp completa (~600 líneas)
```

Es el módulo **más simple**: vista y lógica concentradas en un solo archivo.

Capas

Capa	Archivo	Rol
Vista + Lógica	<code>paint_app.py</code>	Barra superior, toolbar, canvas, herramientas
Datos	En memoria (<code>self.objetos</code>)	Lista de trazos; export PNG vía PIL

Qué ve el usuario

- Barra superior celeste: "🎨 Mini Paint" + botón volver.
- Toolbar**: guardar, cargar, undo/redo, zoom, limpiar, herramientas, formas geométricas, paleta de 16 colores.
- Lienzo blanco** con scrollbars (2000×2000 px).
- Atajos: `Ctrl+Z/Y` , `Ctrl+/-` para zoom.

Integración

- `actualizar_progreso("Dibujo", 0.05)` al abrir.
- Cursor de lápiz aplicado desde `main.py` vía tema.
- Logros solo por porcentaje (25 %, 60 %, 100 %).

Sistema de logros y progreso

Progreso por materia

- Valor de **0.0 a 1.0** (0 % a 100 %) por materia.
- **+5 %** al abrir cualquier módulo desde el menú principal.
- Incrementos adicionales al completar actividades dentro de cada módulo (variable por juego).
- `GestorEstado._sincronizar_logros_porcentaje` desbloquea logros de tipo porcentaje automáticamente.

Tipos de logro

Tipo	Origen del desbloqueo	Ejemplo
porcentaje	Automático al alcanzar umbral	lectura_25 , ajedrez_100
accion	<code>notificar_logro(id)</code> desde un juego	lenguaje_nivel_1 , ajedrez_hierro
general	3 o todas las materias al 100 %	general_3 , general_todas

Persistencia

- Un archivo JSON por usuario en `core/data/usuarios/` .
- Limpieza automática semanal de todos los perfiles (excepto `ultima_limpieza.json`).
- Panel docente (`Ctrl+Shift+A`) para gestión manual.

Cómo añadir un módulo nuevo

1. Crear carpeta en `modulos/` con `crear_frame(parent, on_back_callback)` .
2. Registrar entrada en `ModuleFactory.SUBJECTS` (`import_path` , `callable` , `icono` , `imagen`).
3. Añadir materia en `GestorEstado._estado_default()` .
4. Añadir tema en `MODULO_THEMES` en `main.py` .
5. Añadir tarjeta en `vista_inicio()` (posición en la pirámide).
6. Definir logros en `logros_config.py` si aplica.
7. Añadir imagen de tarjeta en `assets/imagenes/` .

Ejecución y dependencias

Requisitos

```
customtkinter
tkextrafont
Pillow
```

Dependencias adicionales por módulo (no listadas en requirements.txt raíz):

- **Ajedrez:** pygame , python-chess (motor Stockfish incluido en assets).
- **Computación:** pygame .

Arranque

```
pip install -r requirements.txt
python main.py
```

En Linux/ChromeOS también está disponible mundo-escolar.sh .

Atajos de teclado globales

Atajo	Acción
Escape (en módulo)	Volver al menú principal
Escape (en ventana)	Salir de fullscreen
Ctrl+Shift+A	Panel de administración docente

Resumen ejecutivo

Mundo Escolar implementa una arquitectura **modular hub-and-spoke** donde:

- **main.py** orquesta la experiencia global (navegación, usuario, temas, progreso).
- **core/** centraliza estado, carga de módulos, tema y logros mediante **Singleton** y **Factory**.

- Cada módulo en `modulos/` es autónomo y se integra con un único contrato:
`crear_frame(parent, on_back)` .
- La separación **vista / dominio / servicios** es más estricta en Lenguaje y Ajedrez; más concentrada en Dibujo.
- La **vista** instancia clases de dominio, reacciona a eventos del usuario y reporta resultados a `GestorEstado` y `notificar_logro` .
- Lo que el niño ve en pantalla (tarjetas, mapas, tableros, lienzos) es siempre la capa de presentación; la lógica educativa vive detrás en clases sin dependencia de widgets.

Documentación generada para el proyecto Mundo Escolar — plataforma educativa modular para primaria.